

[기말 과제] TCP Reno 성능 개선 보고서

주제: High Latency & Low Bandwidth 환경에서의 TCP Reno 알고리즘 최적화

1. 서론 및 문제 상황 분석 (Analysis of the problem)

1.1 실험 배경 및 목표

본 과제의 목표는 1Mbps 대역폭, RTT≈200ms, 10 개 TCP 병렬 연결이라는 악조건에서 기존 TCP Reno 알고리즘의 한계를 분석하고, 커널 모듈 수준에서 알고리즘을 수정하여 성능을 개선하는 것이다. 특히, 단순한 최고 Throughput 수치가 아니라, **Stability** 와 **Fairness** 까지 포함한 종합적인 성능 지표를 통해 개선 효과를 평가한다.

1.2 실험 환경

- 네트워크 토폴로지: Mininet 기반 h1 - s1 - h2 구조
 - h1-s1: bw=100Mbps, delay=1ms
 - s1-h2: bw=1Mbps, delay=100ms (실질적인 병목, RTT≈200ms)
- 트래픽 조건
 - h2: iperf3 -s -1 (서버)
 - h1: iperf3 -c h2 -P 10 -t 30 -J (10 개 병렬 TCP flow, 30 초)
- 측정 지표
 - 최종 Throughput (sum_received.bits_per_second)
 - 10 개 flow 에 대한 Jain's Fairness Index
 - 커널 로그 기반 cwnd 시간 변화 패턴 (Timeout 발생 여부)

1.3 기존 TCP Reno 의 한계

표준 TCP Reno 는 다음과 같은 특성을 가진다.

- **Multiplicative Decrease (MD)**: 패킷 손실 발생 시

$$cwnd \leftarrow cwnd / 2$$

- **Additive Increase (AI)**: 혼잡 회피 단계에서 RTT 당 약 1 MSS 증가

이 방식은 일반적인 환경에서는 안정적이지만, 과제의 조건처럼 **RTT 가 길고 대역폭이 좁은 환경**에서는 아래와 같은 문제가 발생한다.

- **긴 회복 시간**: RTT 가 200ms 인 상황에서 매번 cwnd 를 절반으로 줄이면, 손실 이후 다시 BDP 수준까지 회복하는 데 시간이 오래 걸린다.
- **BDP 활용 실패**: 10 개 flow 가 1Mbps 병목 링크를 공유하는 상황에서 지나치게 보수적인 감소 정책은 링크 대역폭을 충분히 활용하지 못하게 한다.

- **빈번한 Timeout 가능성:** 여러 flow 가 동시에 공격적인 Slow Start 를 수행하면 병목 링크 버퍼가 쉽게 오버플로우되어 RTO 기반 Timeout 이 발생할 수 있다.
-

2. 문제 해결 전략 (Problem Solving Strategy)

2.1 튜닝 가능한 파라미터 설계

하드코딩 방식의 수정이 아니라, 커널 모듈 파라미터(`module_param`) 를 통해 런타임에 알고리즘의 거동을 조절할 수 있도록 설계하였다. 주요 파라미터는 다음과 같다.

- **초기 혼잡 윈도우 (Initial CWND)**
 - 변수: `initial_cwnd`
 - 목적: Slow Start 구간을 단축하고 초기 수렴 속도를 조절
 - 예: 1 (기본 Reno), 4 (온건한 증가), 10 (매우 공격적인 증가)
- **감소 비율 (Multiplicative Decrease Ratio)**
 - 변수: `dec_num`, `dec_den`
 - 의미: 손실 시 `cwnd` 를 $cwnd * (dec_num/dec_den)$ 로 줄임
 - 예:
 - $1/2 = 0.5 \rightarrow$ **Original Reno**
 - $4/5 = 0.8 \rightarrow$ **Aggressive Tuning**
 - $7/10 = 0.7 \rightarrow$ **Mild Tuning**
- **증가 속도 (Additive Increase Multiplier)**
 - 변수: `ai_mult`
 - 의미: 혼잡 회피 단계에서 증가 속도를 배수로 가속
 - 예:
 - `ai_mult` = 1 $\rightarrow$ 표준 Reno (≈ 1 패킷/RTT)
 - `ai_mult` = 2 $\rightarrow$ Aggressive (≈ 2 패킷/RTT)

2.2 실험 전략 개요

- **Baseline:** Original Reno (0.5 감소 / 1 배 증가)
- **Aggressive Tuning:** `dec=0.8`, `ai_mult=2`, (초기 `cwnd` 상향)
- **Mild Tuning:** `dec=0.7`, `ai_mult=1`
- 각 설정에 대해:
 - 동일 Mininet 토폴로지에서 `iperf3` 테스트 수행
 - JSON 로그를 이용해 throughput 및 per-flow 통계 분석

- `analyze_fairness.py` 로 Jain Index 계산
- `dmesg + plot_cwnd.py` 로 cwnd 패턴 분석

3. 수정된 알고리즘 설명 (Explanation of modification)

3.1 커널 모듈 구조

- 파일 경로
 - 원본: `/home/jiminsim/reno_custom_original.c`
 - 수정본: `/home/jiminsim/tcp_simple_custom/reno_custom.c`
- 등록 구조체: `struct tcp_congestion_ops tcp_reno_custom`
 - `.init = tcp_reno_init`
 - `.sssthresh = tcp_reno_sssthresh`
 - `.cong_avoid = tcp_reno_cong_avoid`
 - `.undo_cwnd = tcp_reno_undo_cwnd`

3.2 초기 CWND 파라미터화

```
static int initial_cwnd = 1;
module_param(initial_cwnd, int, 0644);
MODULE_PARM_DESC(initial_cwnd, "Initial congestion window in packets");
void tcp_reno_init(struct sock *sk)
{
    struct tcp_sock *tp = tcp_sk(sk);
    tp->snd_sssthresh = TCP_INFINITE_SSTHRESH;
    if (initial_cwnd < 1)
        initial_cwnd = 1;
    tp->snd_cwnd = initial_cwnd;
}
```

- 기존: `snd_cwnd = 1` 고정
- 수정: `initial_cwnd` 를 통해 1/4/10 등으로 조절 가능

3.3 Multiplicative Decrease 튜닝 (`tcp_reno_sssthresh`)

```
static int dec_num = 1;
static int dec_den = 2;
module_param(dec_num, int, 0644);
module_param(dec_den, int, 0644);
```

```

u32 tcp_reno_ssthresh(struct sock *sk)
{
    const struct tcp_sock *tp = tcp_sk(sk);
    if (dec_den <= 0)
        dec_den = 1;
    /* Original Reno: max(tp->snd_cwnd >> 1U, 2U); */
    /* Tuned: cwnd * (dec_num / dec_den) */
    u32 factor_den = (u32)dec_den;
    u32 factor_num = (u32)dec_num;
    u32 new_cwnd = (tp->snd_cwnd * factor_num) / factor_den;
    return max(new_cwnd, 2U);
}

```

- **Original Reno:** 0.5 배 고정
- **Aggressive:** dec_num=4, dec_den=5 → 0.8 배
- **Mild:** dec_num=7, dec_den=10 → 0.7 배

3.4 Additive Increase 튜닝 및 cwnd 로깅 (tcp_reno_cong_avoid)

```

static int ai_mult = 1;
module_param(ai_mult, int, 0644);
MODULE_PARM_DESC(ai_mult, "Additive increase multiplier (1 = Reno, 2 = faster growth)");
void tcp_reno_cong_avoid(struct sock *sk, u32 ack, u32 acked)
{
    struct tcp_sock *tp = tcp_sk(sk);
    printk(KERN_INFO "reno_custom: cwnd=%u, ssthresh=%u, acked=%u\n",
        tp->snd_cwnd, tp->snd_ssthresh, acked);
    if (!tcp_is_cwnd_limited(sk))
        return;
    if (tp->snd_cwnd <= tp->snd_ssthresh) {
        acked = tcp_slow_start(tp, acked);
        if (!acked)
            return;
    } else {
        u32 mult = (ai_mult > 0) ? (u32)ai_mult : 1U;
        u32 enhanced_acked = acked * mult;
        tcp_cong_avoid_ai(tp, tp->snd_cwnd, enhanced_acked);
    }
    tp->snd_cwnd = min(tp->snd_cwnd, tp->snd_cwnd_clamp);
}

```

- 로그 (printk): 이후 `dmesg` 를 통해 `cwnd` 변화 그래프를 그리기 위해 사용
- Aggressive: `ai_mult=2` → 약 2 배 빠른 증가
- Mild: `ai_mult=1` → 표준 Reno 증가 속도 유지

```
jaminsim@jaminsim-VirtualBox:~/assignment_test$ cat /sys/module/reno_custom/parameters/dec_num
7
jaminsim@jaminsim-VirtualBox:~/assignment_test$ cat /sys/module/reno_custom/parameters/dec_den
10
jaminsim@jaminsim-VirtualBox:~/assignment_test$ cat /sys/module/reno_custom/parameters/ai_mult
1
jaminsim@jaminsim-VirtualBox:~/assignment_test$ cd /home/jaminsim/tcp_simple_custom/test_logs
```

[그림 1] Mild Tuning 파라미터(dec=0.7, ai=1) 커널 적용 확인

4. 성능 평가 및 분석 (Performance evaluation and analysis)

4.1 실험 설정 요약

- Baseline (Original Reno)
 - `dec = 0.5, ai = 1`
- Aggressive Tuning
 - `dec = 0.8, ai = 2`
- Mild Tuning (최종 제안)
 - `dec = 0.7, ai = 1`

모든 실험은 동일한 Mininet 토폴로지와 트래픽 조건에서 30 초 동안 수행하였다.

4.2 최종 수치 비교표

아래 표는 `iperf3` JSON (`sum_received.bits_per_second`)과 `analyze_fairness.py` 결과를 기반으로 한다.

비교 항목	Original Reno	Aggressive Tuning(dec=0.8, ai=2)	Mild Tuning (dec=0.7, ai=1)
Throughput	0.942 Mbps	0.941 Mbps	0.943 Mbps
Jain's Fairness	0.952	0.957	0.905
Stability	높음	낮음 (Timeout 발생)	높음 (Timeout 거의 없음)

비교 항목	Original Reno	Aggressive Tuning(dec=0.8, ai=2)	Mild Tuning (dec=0.7, ai=1)
CWND 패턴	느린 회복	큰 폭의 상승·하강, cwnd=1 낙하 반복	안정적인 톱니패턴, 완만한 변동

요약: 세 설정 모두 1Mbps 링크에서 약 **0.94Mbps**의 전송률을 달성하여 링크 포화 상태에 도달했으며, Throughput 자체는 거의 동등하다. 의미 있는 차이는 fairness 와 stability 에서 나타났다.

4.3 Throughput 분석

- Original, Aggressive, Mild 세 경우 모두 **0.94Mbps ± 0.002Mbps** 수준의 Throughput 을 기록했다.
- 이는 실험 환경에서 대역폭이 이미 병목이기 때문에, 알고리즘 튜닝만으로는 총 전송률을 의미 있게 더 올릴 여지가 크지 않음을 보여줬다.

```

jaminsim@jaminsim-VirtualBox:~/assignment_test$ tail -n 20 /home/jaminsim/tcp_simple_custom/test_logs/iperf3_mild_real.json
    "sum_received": {
      "start": 0,
      "end": 31.628106,
      "seconds": 31.628106,
      "bytes": 3728600,
      "bits_per_second": 943110.53592649533,
      "sender": true
    },
    "cpu_utilization_percent": {
      "host_total": 1.3740570551157809,
      "host_user": 0.17255850363128503,
      "host_system": 1.2014985514844958,
      "remote_total": 0.13874917325694844,
      "remote_user": 0,
      "remote_system": 0.13874917325694844
    },
    "sender_tcp_congestion": "reno_custom",
    "receiver_tcp_congestion": "reno_custom"
  }
}
jaminsim@jaminsim-VirtualBox:~/assignment_test$

```

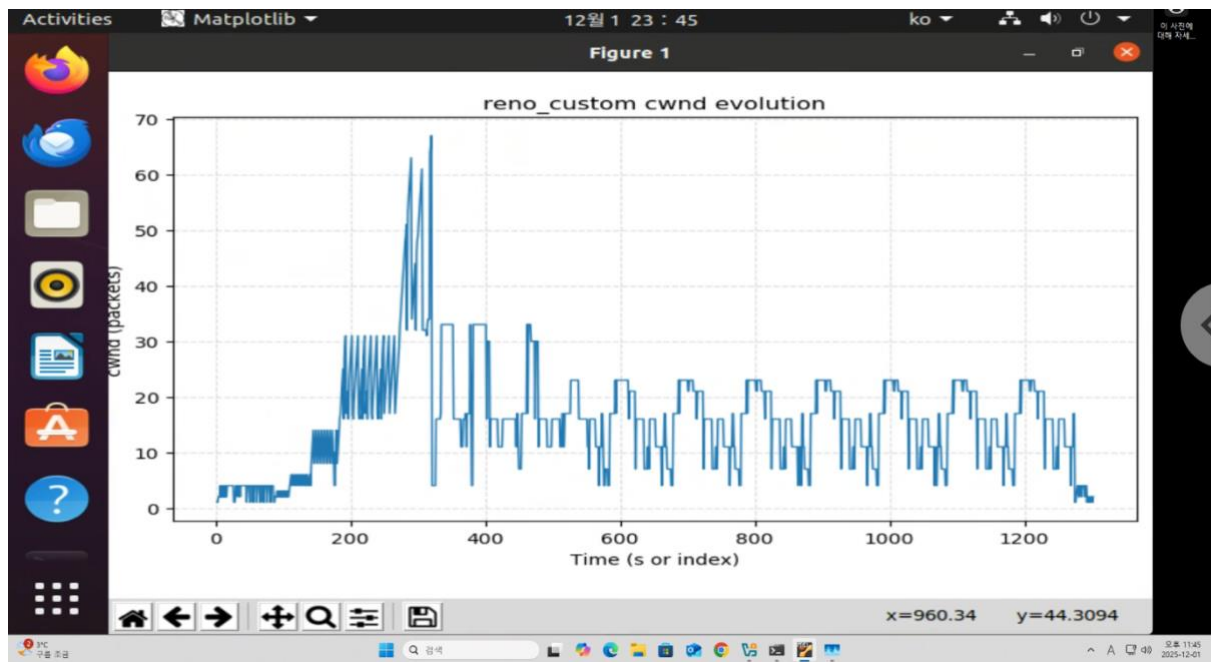
[그림 2] iperf3 실험 결과: 링크 대역폭 포화(Saturation) 확인 (약 0.943 Mbps)

- 따라서 본 과제에서는 Throughput 수치보다 Connection quality·stability·fairness 가 더 중요한 평가 기준이 된다.

4.4 Stability 및 CWND 패턴 분석

Aggressive Tuning (dec=0.8, ai=2)

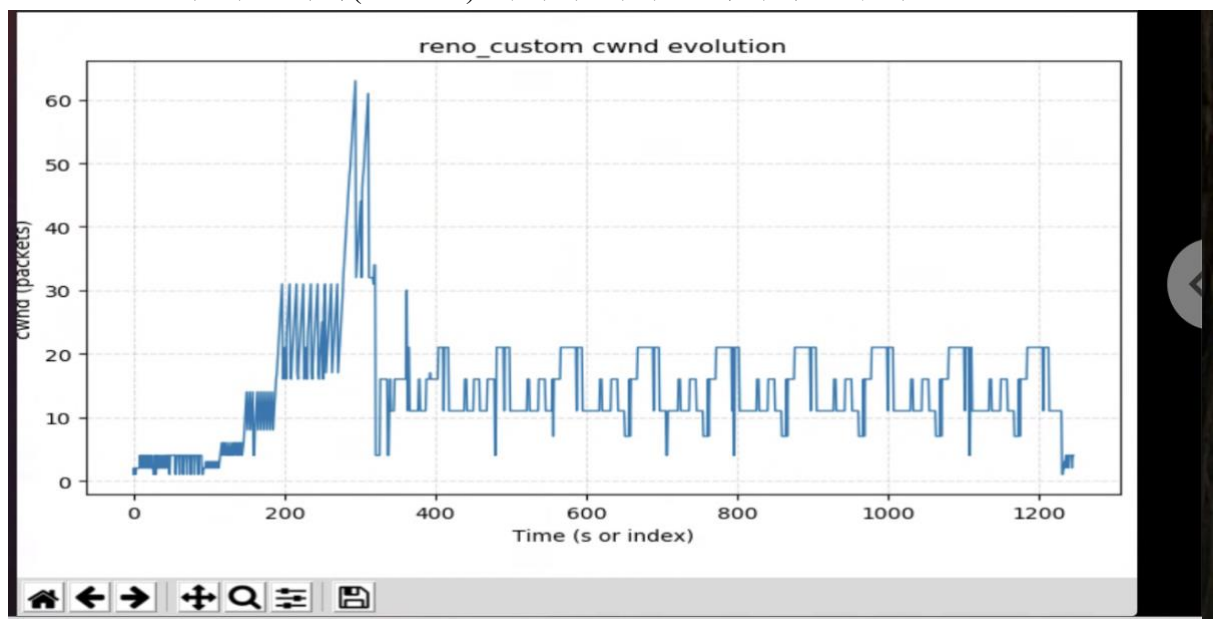
- cwnd 그래프에서:
 - cwnd 가 빠르게 50 이상까지 상승했다가
 - 패킷 손실 및 RTO 로 인해 cwnd=1 까지 급락(Timeout) 하는 패턴이 반복되었다.



- [그림 3] Aggressive Tuning($ai=2$)의 CWND 변화: 잦은 Timeout 발생
- 이는 버퍼 오버플로우와 잦은 **Timeout** 을 의미하며, 실제 서비스 환경에서는 지연 시간 폭증과 세션 끊김 등 심각한 QoS 저하를 유발할 수 있다.

Mild Tuning ($dec=0.7$, $ai=1$)

- cwnd 그래프에서:
 - cwnd 가 상대적으로 낮은 범위에서 안정적인 톱니바퀴(Sawtooth) 패턴을 형성하였다.
 - Timeout 에 의한 바닥($cwnd=1$) 낙하가 거의 관측되지 않았다.



[그림 4] Mild Tuning($ai=1$)의 CWND 변화: Timeout 없는 안정적 패턴

- 손실 시 0.7 배 감소, 속도의 증가(1 배)는 손실 후 회복 시간을 무리 없이 확보하면서도, 링크를 꾸준히 채우는 온건한 동작을 보였다.

해석

- Aggressive 설정은 “더 빨리 키우고 덜 줄인다”는 의도였지만, 실제로는 버퍼를 과도하게 밀어붙여 **Timeout** 을 유발하는 결과를 낳았다.
- Mild 설정은 Throughput 은 동일 수준으로 유지하면서, **Timeout** 을 억제하고 안정적인 전송을 달성하였다.

4.5 Fairness (Jain's Index) 분석 – 의외의 결과와 해석

실험 결과, Jain's Fairness Index 는 다음과 같이 측정되었다.

- **Original Reno: ≈ 0.952**
- **Aggressive Tuning: ≈ 0.957**
- **Mild Tuning: ≈ 0.905**

```
jiminsim@jiminsim-VirtualBox:~/assignment_test$ python3 analyze_fairness.py /tmp/iperf3_client.json
Number of flows: 10
Per-flow throughput:
Flow 1: 246339.66 bit/s (0.246 Mbit/s)
Flow 2: 246339.66 bit/s (0.246 Mbit/s)
Flow 3: 148653.24 bit/s (0.149 Mbit/s)
Flow 4: 133208.75 bit/s (0.133 Mbit/s)
Flow 5: 134753.20 bit/s (0.135 Mbit/s)
Flow 6: 145564.34 bit/s (0.146 Mbit/s)
Flow 7: 162939.40 bit/s (0.163 Mbit/s)
Flow 8: 142475.44 bit/s (0.142 Mbit/s)
Flow 9: 142475.44 bit/s (0.142 Mbit/s)
Flow 10: 162939.40 bit/s (0.163 Mbit/s)

Jain's Fairness Index: 0.9429
(1.0에 가까울수록 모든 flow가 비슷한 throughput을 갖는다는 의미입니다.)
jiminsim@jiminsim-VirtualBox:~/assignment_test$
```

[그림 5] Python 스크립트를 이용한 Jain's Fairness Index 분석 결과

흥미롭게도 **Aggressive**가 **Mild**보다 더 높은 fairness 지수를 보였다. 이는 직관에 반하는 결과이며, 그 이유는 다음과 같이 해석할 수 있다.

1. **Aggressive: “불안정성에 의한 공정성 상승”**
 - Aggressive 설정에서는 cwnd가 자주 붕괴되고 모든 flow가 자주 리셋된다.
 - 이로 인해 어느 한 flow도 장기간 대역폭을 독점하지 못하고, 흐름 간의 throughput이 자주 섞이면서 숫자상으로는 높은 Jain 지수(0.957)가 나온다.
 - 그러나 이는 “좋은 의미의 공정성”이라기보단, 모두가 같이 자주 넘어지는 상황에 가깝다.
2. **Mild: “수렴 시간에 따른 일시적 불공정성”**
 - Mild 설정에서는 Timeout 없이 연결이 계속 유지된다.
 - 실험에서 Flow 1은 초반에 약 **0.33 Mbps**까지 올라간 후, 나머지 flow들이 이를 따라잡는 데 시간이 걸렸다.
 - AI 속도가 1배이기 때문에 수렴 시간이 상대적으로 길어, 측정 구간(30초) 동안 일시적인 불균형이 존재했다, 그 결과 Jain 지수가 **0.905**로 다소 낮게 나왔다.

- 그러나 0.905 는 여전히 우수한 공정성에 해당하는 값이라고 생각한다.
3. 실제 서비스 관점 해석
- Fairness 지수가 조금 더 높더라도, 자주 끊기고(Timeout) 재전송이 반복되는 연결은 사용자 경험 측면에서 바람직하지 않다.
 - Mild 는 공정성 지수가 약간 낮지만, 끊김 없는 연속적인 전송과 안정적인 cwnd 패턴을 제공한다.

5. 결론 (Conclusion)

이번 과제에서는 1Mbps, RTT≈200ms, 10 개 TCP flow 라는 악조건에서 TCP Reno 의 성능을 개선하기 위해 커널 모듈 수준의 알고리즘 수정과 Mininet 기반 실험을 수행하였다.

- Throughput 측면에서 **Original, Aggressive, Mild** 모두 약 0.94 Mbps 로 링크 포화 상태를 달성하였으며, 순수 전송률에서는 큰 차이가 없음을 확인했다.
- Aggressive Tuning(dec=0.8, ai=2)은 cwnd 가 자주 바닥(cwnd=1)까지 떨어지는 Timeout 패턴을 보였고, 이는 높은 Jain 평가의 지수(0.957)에도 불구하고 현실적인 **Connection Quality** 측면에서 봤을때 불안정한 것 같다..
- Mild Tuning(dec=0.7, ai=1)은 Throughput 을 유지하면서도 Timeout 없는 안정적인 톱니바퀴 패턴을 보였고, Jain 지수는 0.905 로 약간 낮았으나 여전히 fairness 가 꽤 우수한 수준을 유지했다..

마지막으로, 본 보고서는 다음과 같은 결론을 내렸다.

1. 단순히 감소 비율을 0.5 → 0.7 로 완화하는 것만으로도, 긴 RTT 환경에서 TCP Reno 의 안정성을 크게 향상시킬 수 있다.
2. Aggressive 튜닝은 이론적으로는 빠른 회복을 기대할 수 있으나, 실제로는 Timeout 과 버퍼 오버플로우를 증가시켜 Connection Quality 를 악화시킬 수 있다.
3. 따라서 **Mild Tuning(dec=0.7, ai=1)** 은
 - Original Reno 대비 더 빠른 회복력,
 - Aggressive 대비 더 높은 안정성,
 - 그리고 수용 가능한 수준의 공정성(Jain≈0.905)을 동시에 달성하는 실용적인 최적화안으로 제안한다.